An Introduction to the R Programming Language

Part Two

Presented by: Samuel Callisto, PhD
Group Leader PKPD, Metrum Research Group
Adjunct Assistant Prof, ECP UMN
2 October 2026

Homework #1: Building a data.frame

- Create a data.frame matching the table shown
  - hint: type `?rep()` into the console to see help for the replicate function
- Save the output to an object named "dat"
- Verify that the structure and contents of dat are correct
- Save "dat" to disk in your working directory with the name "output.csv"

ID	TIME	DV	SEX	WT	ROW
1	1	2	M	70	1
1	6	4	M	70	2
1	24	6	M	72	3
2	1	2	F	64	4
2	6	4	F	64	5
2	24	6	F	63	6
3	1	2	M	98	7
3	6	4	M	98	8
3	24	6	M	98	9

Homework #1 Solutions

```
## create dat object
n_id <- 3
dat <- data.frame(
  ID=sort(rep(1:n_id, 3)),
  TIME=rep(c(1,6,24), n_id),
  DV = rep(seq(2,6,2), n_id),
  SEX = c(rep("M",3),
          rep("F",3),
          rep("M",3)),
  WT = c(70,70,72,
         64,64,63,
         98,98,98),
  ROW = seq(1,9)
)
```

```
## print to Console
dat

## verify class
class(dat)

## inspect structure
str(dat)

## write to file
readr::write_csv(dat, "output.csv")
```



Homework #2: Modifying an existing data set

- Import mad-nonmem.csv
- Filter out commented rows (i.e. in column C, rows equal to “C”)
- Create new columns in the data set
  - TIMED = time in days, rounded to the nearest hundredth
  - SEX_c = recode SEX column from 0/1 to M/F
  - DOSE_f = factor of doses in ascending order
- Drop all columns between TAD and ADDL (inclusive), and DTTM
- Move TIMED next to the TIME column

Homework #2 Solution

```
dat <- readr::read_csv("mad-nonmem.csv")

## make changes to data set, save as new object
dat_filt <- dat %>%
  filter(C != "C") %>%
  mutate(
    TIMED = round(TIME/24, 2),
    SEX_c = if_else(SEX == 0, "M", "F"),
    DOSE_f = factor(DOSE, levels = c(0.5,2.5,10,25))
  ) %>%
  select(-c(TAD:ADDL), DTTM) %>%
  relocate(TIMED, .after=TIME)

dat_filt
```



Homework Problem #3: Restructuring a data set

- Create a new data frame using a pivot_* function to create a three-column data set, where the first column is the ID, the second column is the name of a covariate, and the third column is the value of a covariate

Homework #3 Solution

```
dat_wide <- dat_filt %>%
  distinct(ID,SEX,AGE,WT) %>%
  group_by(ID) %>%
  slice(1) %>%
  ungroup() %>%
  filter(ID %in% c(1,4,7))
```

	ID	name	value
1	1	SEX	1
2	1	AGE	45
3	1	WT	77.2
4	4	SEX	1
5	4	AGE	36
6	4	WT	72.3
7	7	SEX	2
8	7	AGE	64
9	7	WT	119

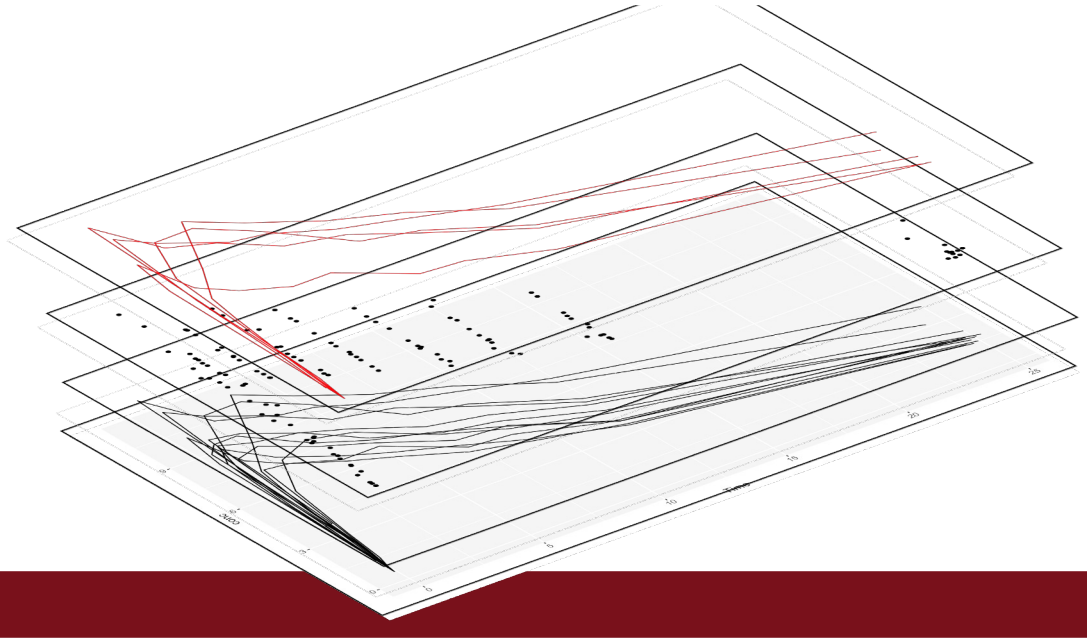
```
dat_long <- dat_wide %>%
  pivot_longer(cols=SEX:WT, names_to = "name",
               values_to = "value")
dat_long
```



ggplot

ggplot: a modern system for data visualization

- The grammar of graphics (gg) consists of the sum of several independent layers of objects that are combined using + to construct the final graph



Layers are stacked sequentially to create the final plot

```
ggplot(aes()) +
    geom_line() +
    geom_point() +
    geom_line(col="red")
```

Elements of ggplot

Element	Code	Use
data	data =	Raw data for plotting
aesthetics:	aes()	Aesthetics of the geoms and stats – control **color, size, shape, fill**, etc and how it is mapped to underlying data
geometries:	geom_<type>	Shapes to represent the data +settings unrelated to data
scales	scale_	Maps the coordinate system for the geoms
Statistical transformation	stat_	transforms the data, typically by summarizing it in some manner.
Additional Customizations	theme	Control axis, background, tick settings (size, color, etc)



https://rstudio.github.io/cheatsheets/

Data visualization with ggplot2 : : CHEAT SHEET

Basics

ggplot2 is based on the **grammar of graphics**, the idea that you can build every graph from the same components: a **data** set, a **coordinate system**, and **geoms**—visual marks that represent data points.



To display values, map variables in the data to visual properties of the geom (**aesthetics**) like **size**, **color**, and **x** and **y** locations.



Complete the template below to build a graph.

```
ggplot (data = <DATA>) +                                   required
<GEOM_FUNCTION>(mapping = aes(<MAPPINGS>),
stat = <STAT>, position = <POSITION>) +                    Not required, sensible defaults supplied
<COORDINATE_FUNCTION> +
<FACET_FUNCTION> +
<SCALE_FUNCTION> +
<THEME_FUNCTION>
```

ggplot(data = mpg, **aes**(x = cty, y = hwy)) Begins a plot that you finish by adding layers to. Add one geom function per layer.

last_plot() Returns the last plot.

ggsave("plot.png", width = 5, height = 5) Saves last plot as 5' x 5' file named "plot.png" in working directory. Matches file type to file extension.

Aes

Common aesthetic values.

color and **fill** - string ("red", "#RRGGBB")

linetype - integer or string (0 = "blank", 1 = "solid", 2 = "dashed", 3 = "dotted", 4 = "dotdash", 5 = "longdash", 6 = "twodash")

lineend - string ("round", "butt", or "square")

linejoin - string ("round", "mitre", or "bevel")

size - integer (line width in mm)

shape - integer/shape name or a single character ("a")



posit™

Geoms

Use a geom function to represent data points, use the geom's aesthetic properties to represent variables. Each function returns a layer.

GRAPHICAL PRIMITIVES

a <- ggplot(economics, aes(date, unemploy))
b <- ggplot(seals, aes(x = long, y = lat))

a + geom_blank() and **a + expand_limits()** Ensure limits include values across all plots.

b + geom_curve(aes(yend = lat + 1, xend = long + 1), curvature = 1) - x, xend, y, yend, alpha, angle, color, curvature, linetype, size

a + geom_path(lineend = "butt", linejoin = "round", linemitre = 1) x, y, alpha, color, group, linetype, size

a + geom_polygon(aes(alpha = 50)) - x, y, alpha, color, fill, group, subgroup, linetype, size

b + geom_rect(aes(xmin = long, ymin = lat, xmax = long + 1, ymax = lat + 1)) - xmax, xmin, ymax, ymin, alpha, color, fill, linetype, size

a + geom_ribbon(aes(ymin = unemploy - 900, ymax = unemploy + 900)) - x, ymax, ymin, alpha, color, fill, group, linetype, size

LINE SEGMENTS

common aesthetics: x, y, alpha, color, linetype, size

b + geom_abline(aes(intercept = 0, slope = 1))
b + geom_hline(aes(yintercept = lat))
b + geom_vline(aes(xintercept = long))

b + geom_segment(aes(yend = lat + 1, xend = long + 1))
b + geom_spoke(aes(angle = 1:1155, radius = 1))

ONE VARIABLE continuous

c <- ggplot(mpg, aes(hwy)); c2 <- ggplot(mpg)

c + geom_area(stat = "bin") x, y, alpha, color, fill, linetype, size

c + geom_density(kernel = "gaussian") x, y, alpha, color, fill, group, linetype, size, weight

c + geom_dotplot() x, y, alpha, color, fill

c + geom_freqpoly() x, y, alpha, color, group, linetype, size

c + geom_histogram(binwidth = 5) x, y, alpha, color, fill, linetype, size, weight

c2 + geom_qq(aes(sample = hwy)) x, y, alpha, color, fill, linetype, size, weight

discrete

d <- ggplot(mpg, aes(fl))

d + geom_bar() x, alpha, color, fill, linetype, size, weight

TWO VARIABLES both continuous

e <- ggplot(mpg, aes(cty, hwy))

e + geom_label(aes(label = cty), nudge_x = 1, nudge_y = 1) - x, y, label, alpha, angle, color, family, fontface, hjust, lineheight, size, vjust

e + geom_point() x, y, alpha, color, fill, shape, size, stroke

e + geom_quantile() x, y, alpha, color, group, linetype, size, weight

e + geom_rug(sides = "bl") x, y, alpha, color, linetype, size

e + geom_smooth(method = lm) x, y, alpha, color, fill, group, linetype, size, weight

e + geom_text(aes(label = cty), nudge_x = 1, nudge_y = 1) - x, y, label, alpha, angle, color, family, fontface, hjust, lineheight, size, vjust

one discrete, one continuous

f <- ggplot(mpg, aes(class, hwy))

f + geom_col() x, y, alpha, color, fill, group, linetype, size

f + geom_boxplot() x, y, lower, middle, upper, ymax, ymin, alpha, color, fill, group, linetype, shape, size, weight

f + geom_dotplot(binaxis = "y", stackdir = "center") x, y, alpha, color, fill, group

f + geom_violin(scale = "area") x, y, alpha, color, fill, group, linetype, size, weight

both discrete

g <- ggplot(diamonds, aes(cut, color))

g + geom_count() x, y, alpha, color, fill, shape, size, stroke

e + geom_jitter(height = 2, width = 2) x, y, alpha, color, fill, shape, size

continuous bivariate distribution

h <- ggplot(diamonds, aes(carat, price))

h + geom_bin2d(binwidth = c(0.25, 500)) x, y, alpha, color, fill, linetype, size, weight

h + geom_density_2d() x, y, alpha, color, group, linetype, size

h + geom_hex() x, y, alpha, color, fill, size

continuous function

i <- ggplot(economics, aes(date, unemploy))

i + geom_area() x, y, alpha, color, fill, linetype, size

i + geom_line() x, y, alpha, color, group, linetype, size

i + geom_step(direction = "hv") x, y, alpha, color, group, linetype, size

visualizing error

df <- data.frame(grp = c("A", "B"), fit = 4:5, se = 1:2)
j <- ggplot(df, aes(grp, fit, ymin = fit - se, ymax = fit + se))

j + geom_crossbar(fatten = 2) - x, y, ymax, ymin, alpha, color, fill, group, linetype, size

j + geom_errorbar() - x, ymax, ymin, alpha, color, group, linetype, size, width Also **geom_errorbarh()**.

j + geom_linerange() x, ymin, ymax, alpha, color, group, linetype, size

j + geom_pointrange() - x, y, ymin, ymax, alpha, color, fill, group, linetype, shape, size

maps

data <- data.frame(murder = USArrests$Murder, state = tolower(rownames(USArrests)))
map <- map_data("state")
k <- ggplot(data, aes(fill = murder))

k + geom_map(aes(map_id = state), map = map) **+ expand_limits**(x = map$long, y = map$lat) map_id, alpha, color, fill, linetype, size

THREE VARIABLES

seals$z <- with(seals, sqrt(delta_long^2 + delta_lat^2)); l <- ggplot(seals, aes(long, lat))

l + geom_contour(aes(z = z)) x, y, z, alpha, color, group, linetype, size, weight

l + geom_contour_filled(aes(fill = z)) x, y, alpha, color, fill, group, linetype, size, subgroup

l + geom_raster(aes(fill = z), hjust = 0.5, vjust = 0.5, interpolate = FALSE) x, y, alpha, fill

l + geom_tile(aes(fill = z)) x, y, alpha, color, fill, linetype, size, width

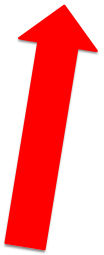
RCC BY SA Posit Software, PBC • info@posit.co • posit.co • Learn more at **ggplot2.tidyverse.org** • ggplot2 3.3.5 • Updated: 2021-08

```
ggplot(data=Theoph, aes(x = Time, y = conc))
```



Underlying data

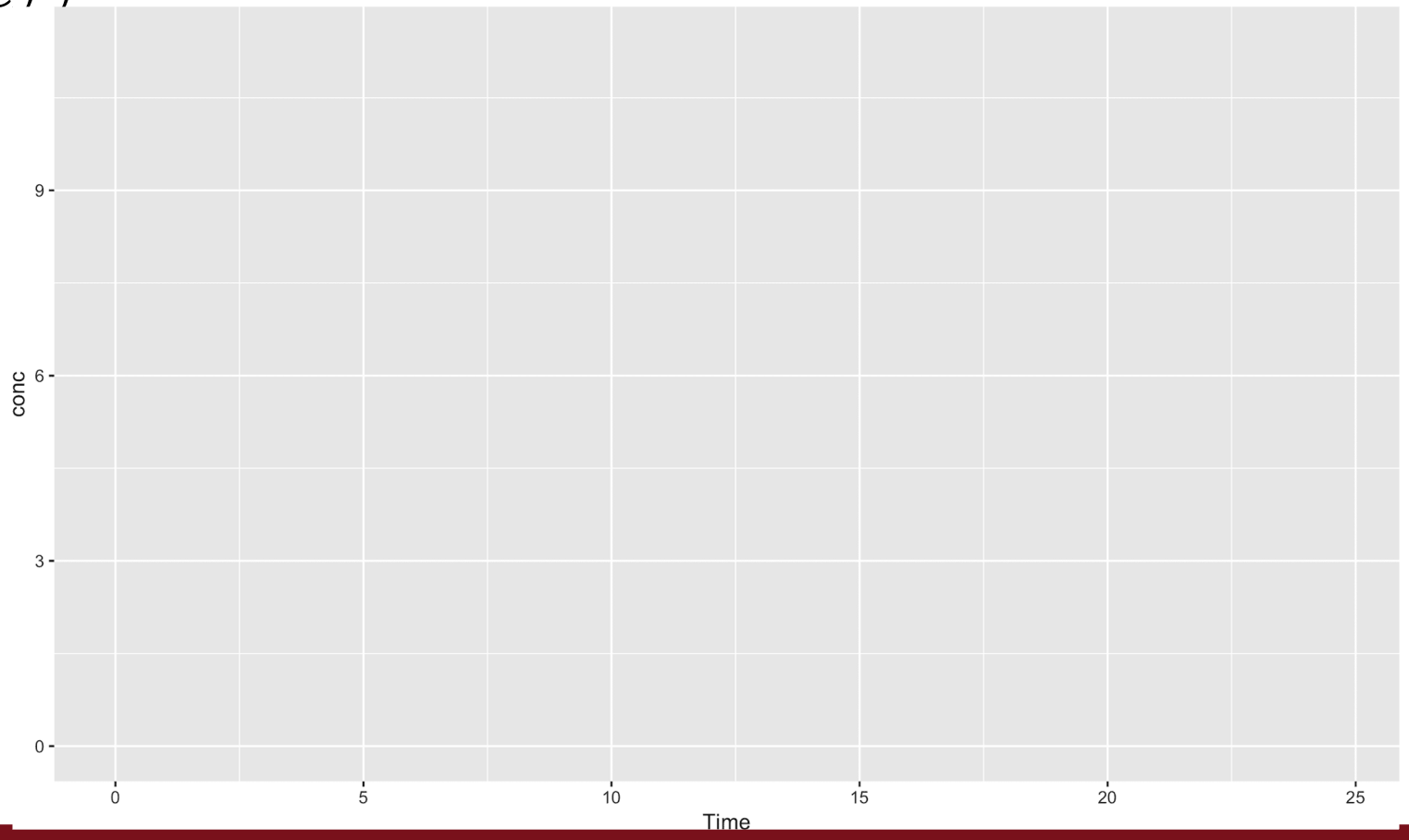




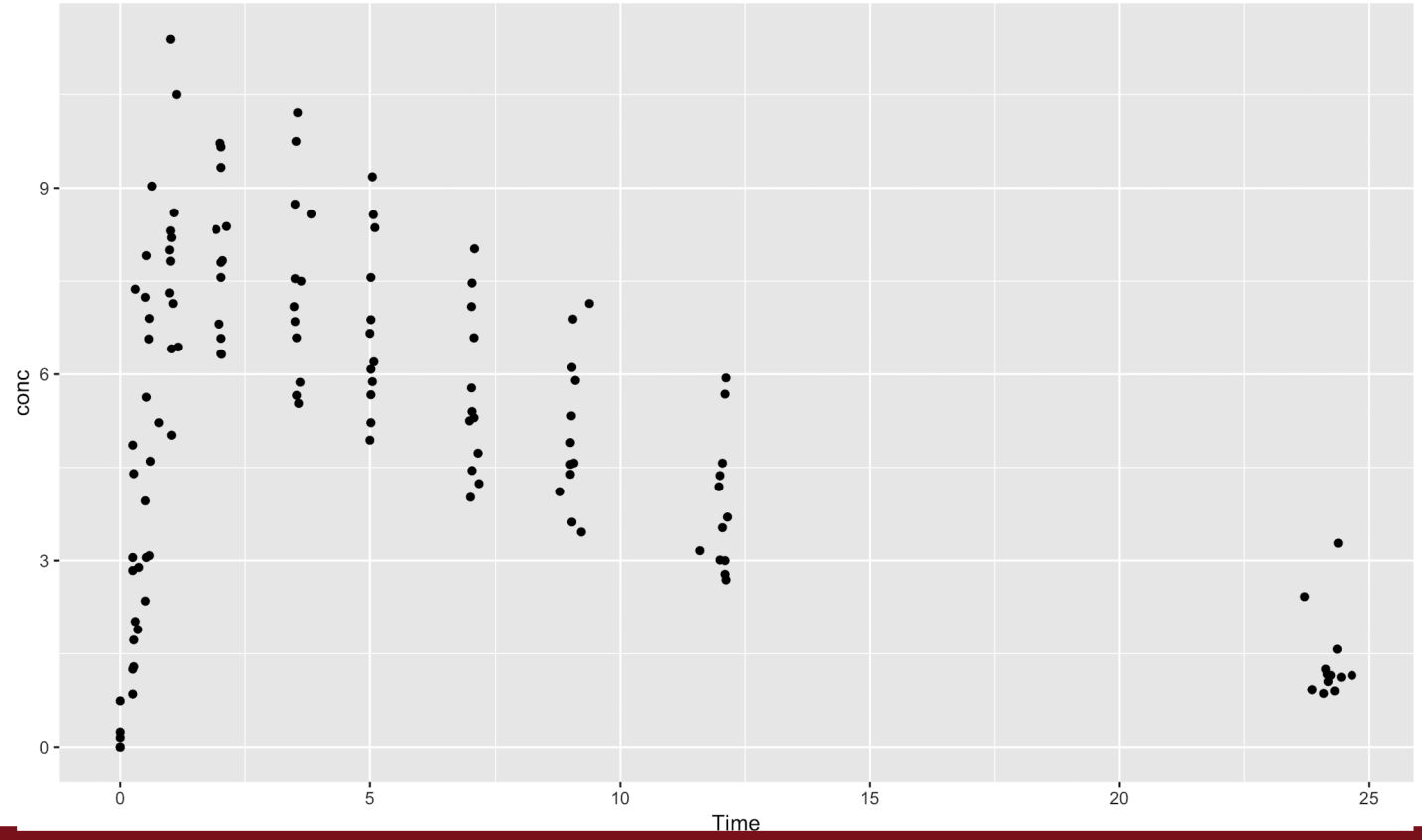
Data columns and their associated aesthetic 'mappings'

Subject	Wt	Dose	Time	conc
1	79.6	4.02	0	0.74
1	79.6	4.02	0.25	2.84
1	79.6	4.02	0.57	6.57
1	79.6	4.02	1.12	10.5
1	79.6	4.02	2.02	9.66
1	79.6	4.02	3.82	8.58

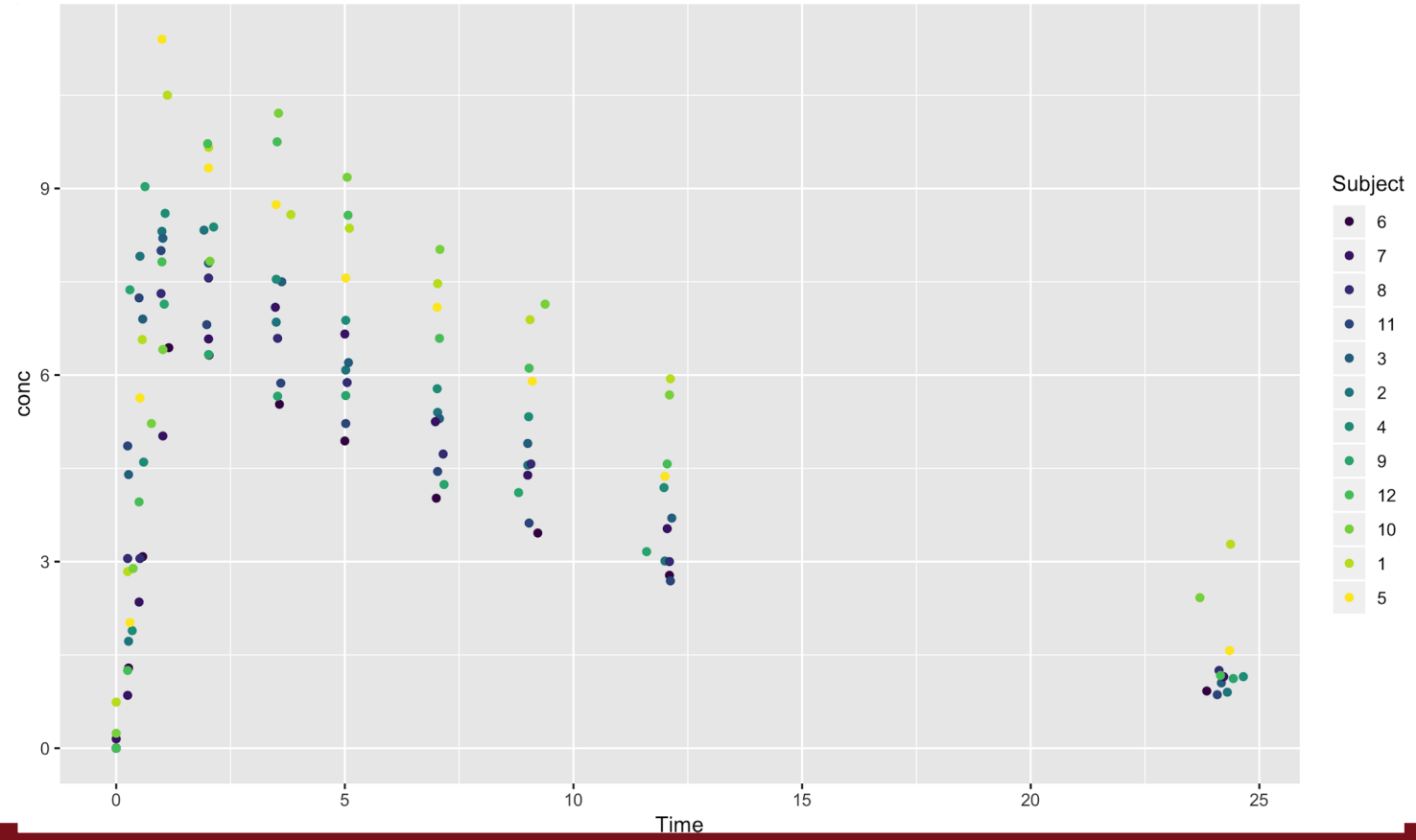
```
ggplot(data=Theoph, aes(x = Time, y =
conc))
```



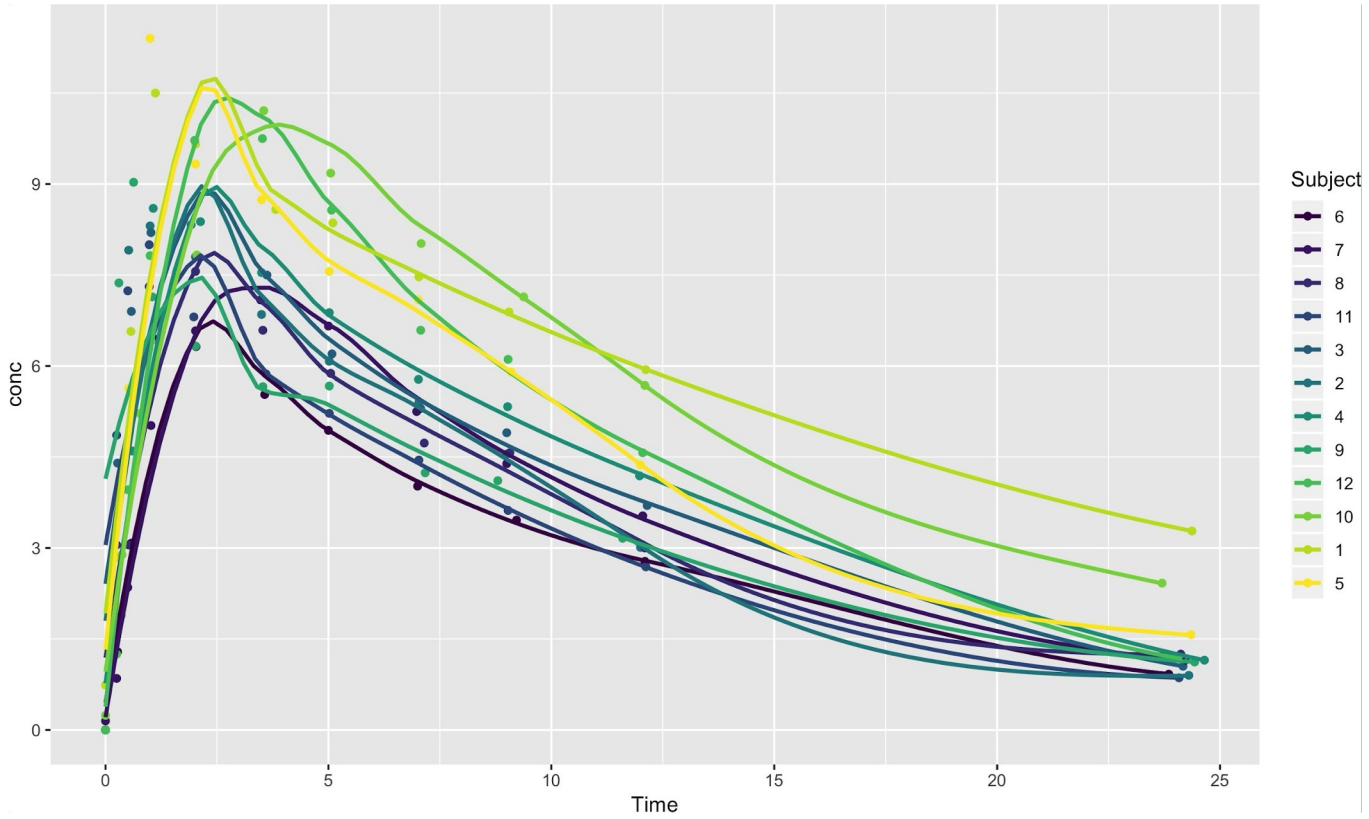
```
ggplot(data=Theoph, aes(x = Time, y = conc)) +
    geom_point()
```



```
ggplot(data=Theoph, aes(x = Time, y = conc,
color=Subject)) + geom_point()
```



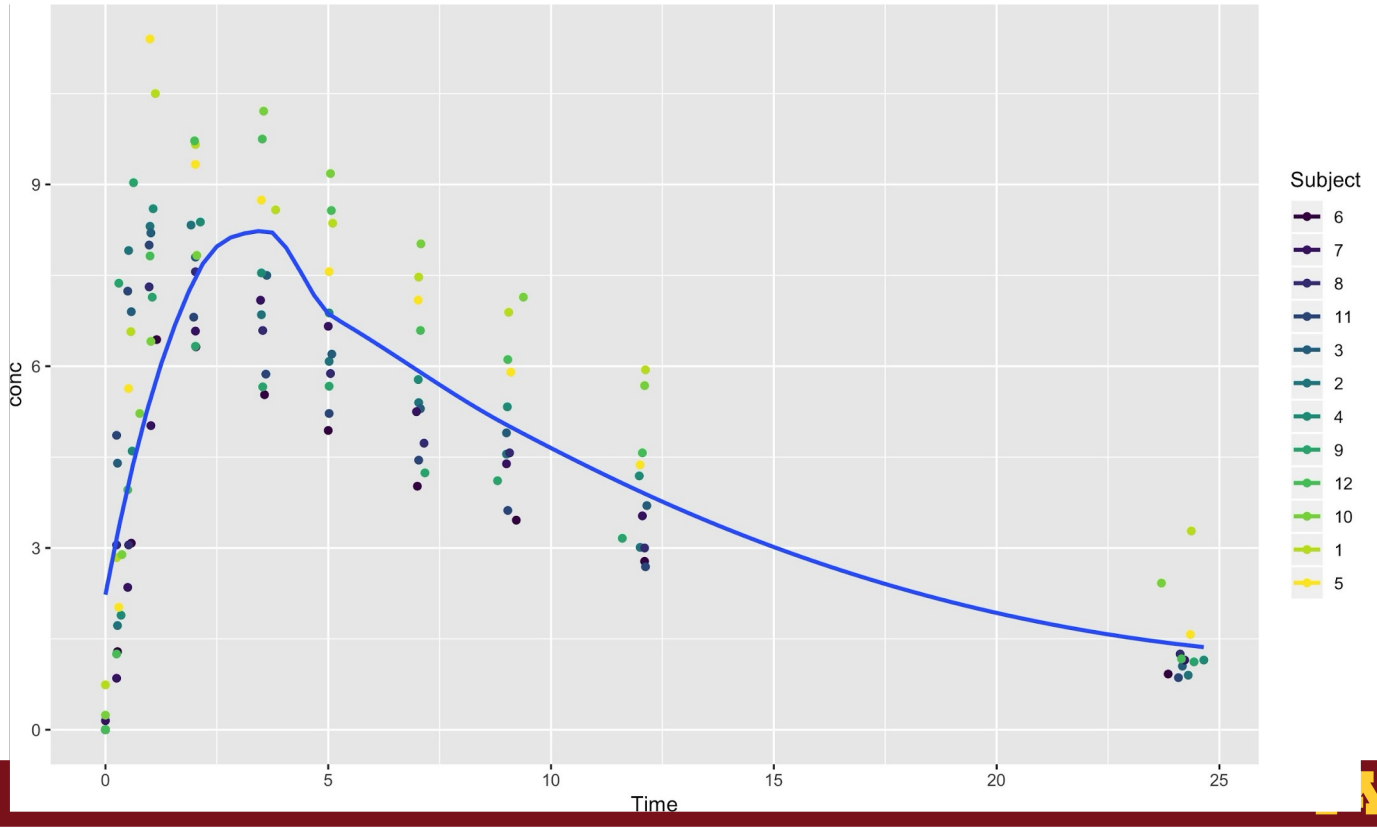
```
ggplot(data=Theoph,aes(x=Time, y=conc,
color=Subject))+ geom_point()+ geom_smooth(se=FALSE)
```



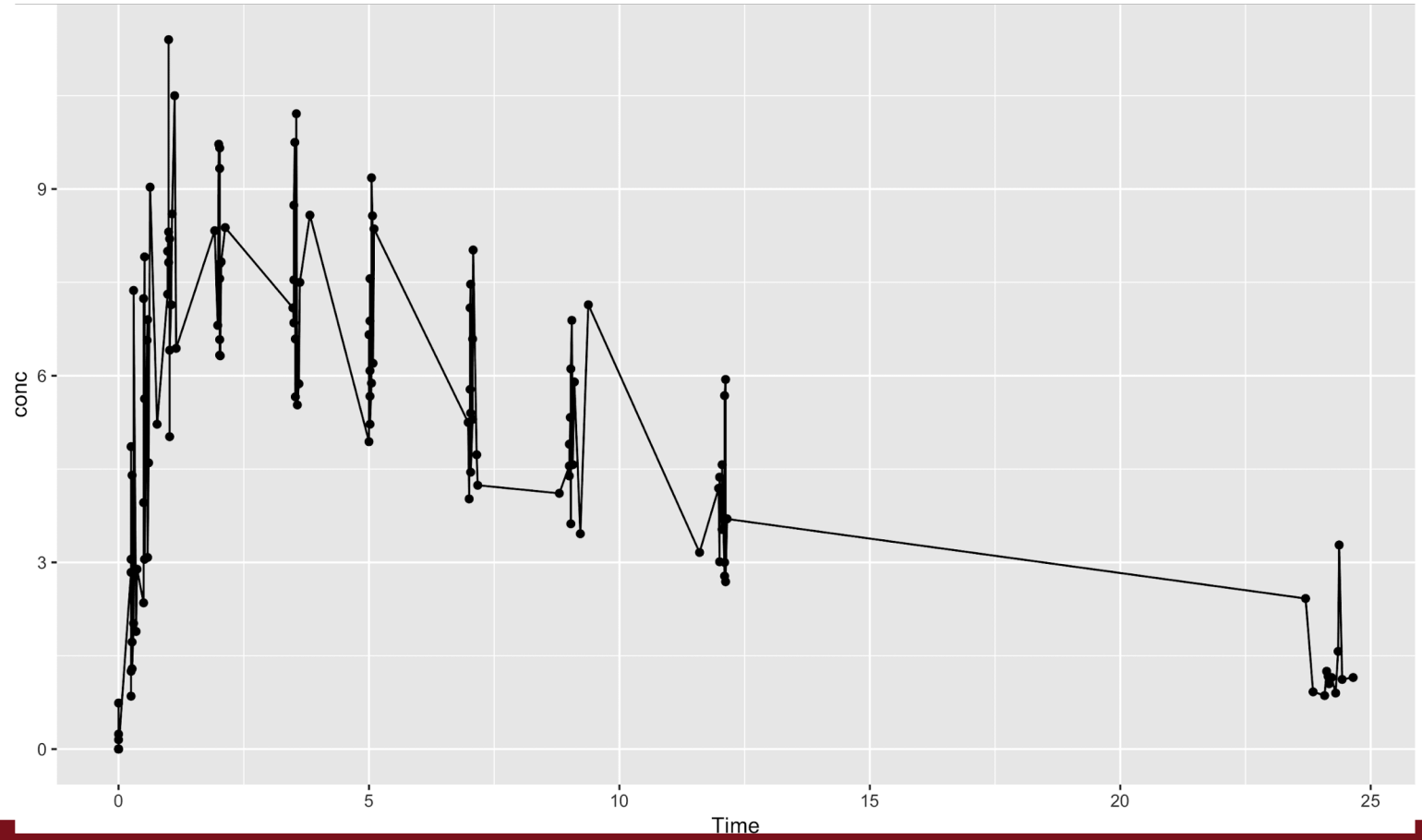
Propagating aesthetic properties

```
ggplot(data=Theoph,aes(x=Time,y=conc, color = Subject))+
  geom_point()+
  geom_smooth(se=FALSE, aes(color = NULL))
```

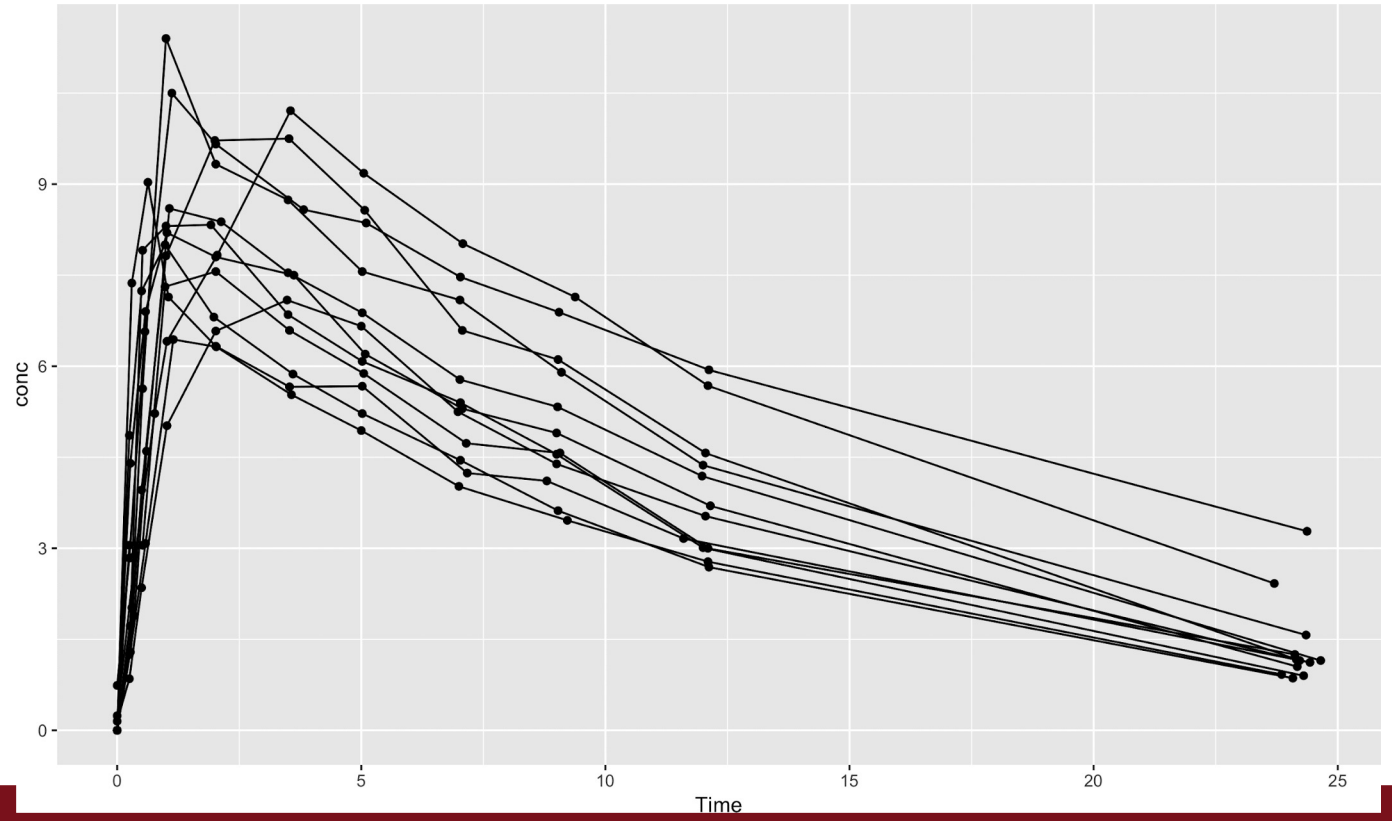
data and aes() are inherited from ggplot call, but can also be assigned individually for each geom



```
ggplot(data=Theoph, aes(x = Time, y = conc)) +
geom_point() + geom_line()
```



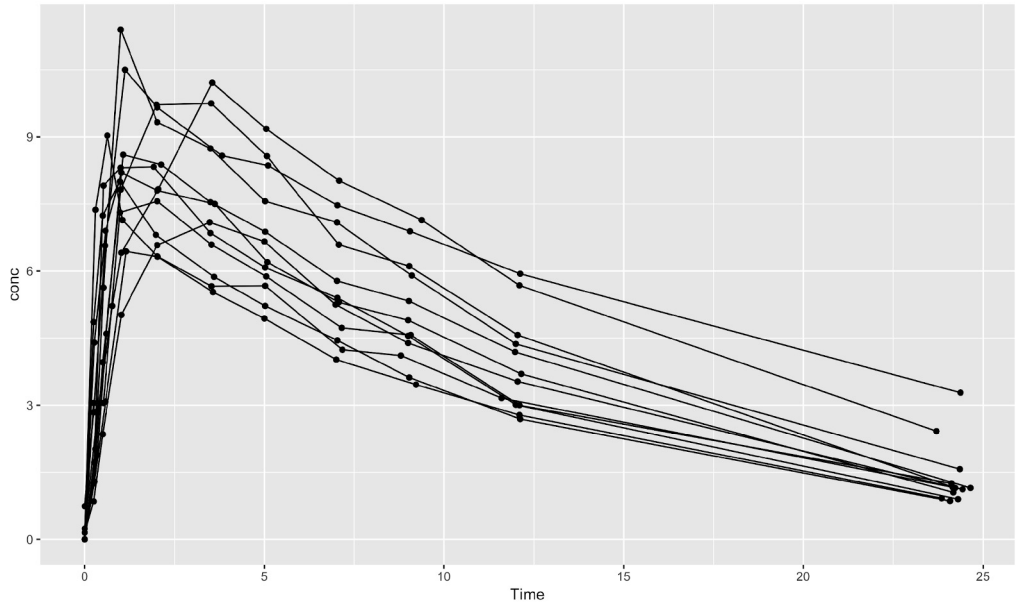
```
ggplot(data=Theoph, aes(x = Time, y = conc,
    group = Subject)) +
    geom_point() + geom_line()
```



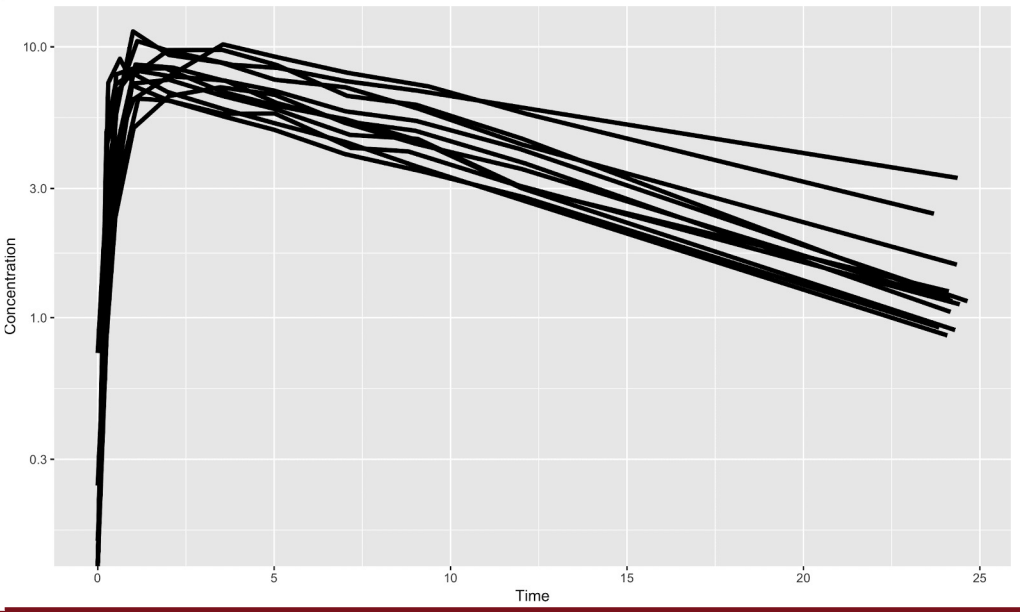
Objects can be saved and more layers added

```
conc_time <- ggplot(data = Theoph,
    aes(x = Time, y = conc, group = Subject)
) + geom_line()
```

```
conc_time
```

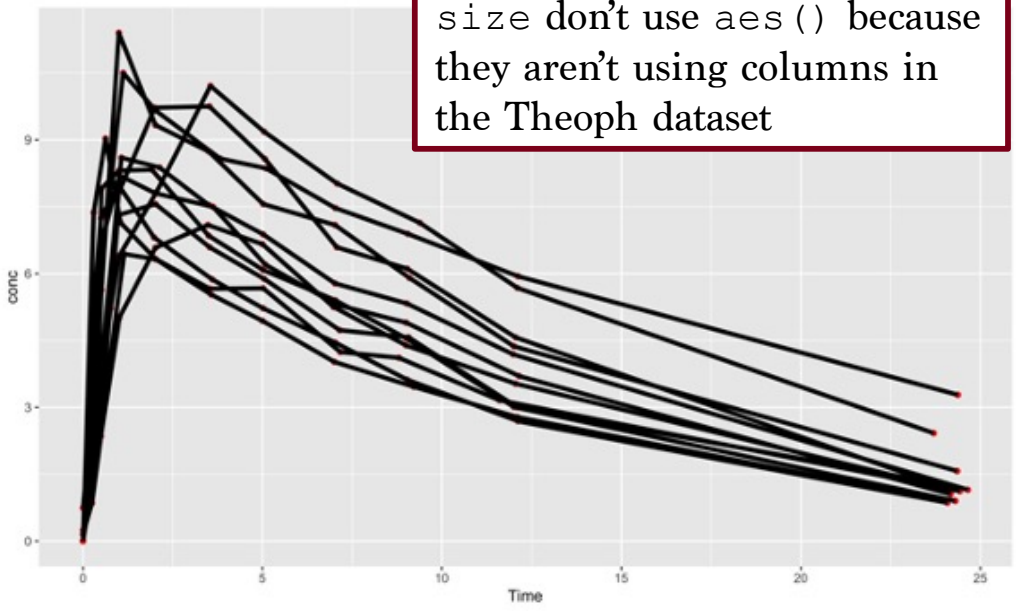



```
conc_time +
  scale_y_log10() +
  ylab("Concentration")
```

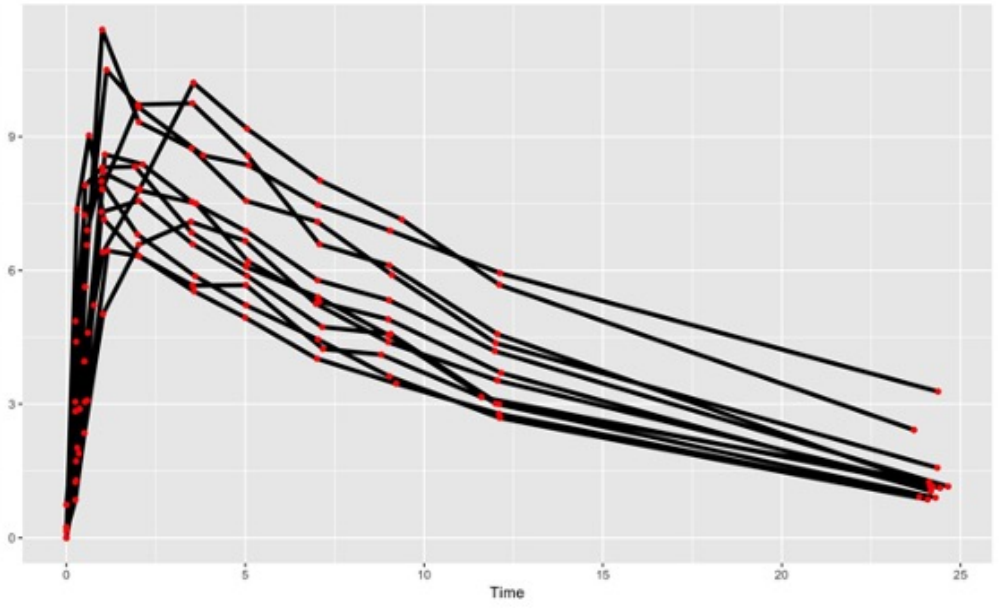


Order matters visually!

```
ggplot(data = Theoph, aes(x = Time,
  y = conc, group = Subject)) +
  geom_point(color = 'red') +
  geom_line(size = 1.5)
```



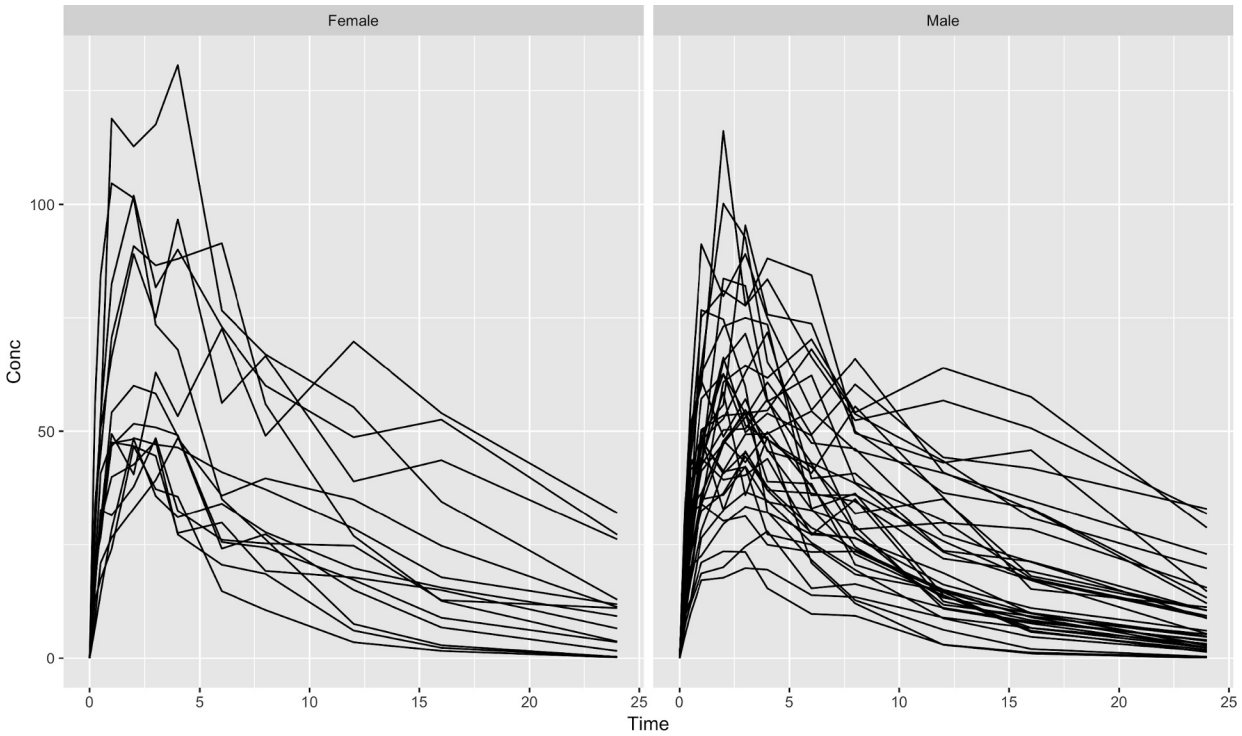

```
ggplot(data = Theoph, aes(x = Time,
  y = conc, group = Subject)) +
   geom_line(size = 1.5) +
   geom_point(color = 'red')
```



Notice here that `color` and `size` don't use `aes()` because they aren't using columns in the Theoph dataset

Facets

```
ggplot(sd_oral_richpk, aes(x = Time, y = Conc)) +
    geom_line(aes(group = ID)) +
    facet_wrap(~Gender)
```

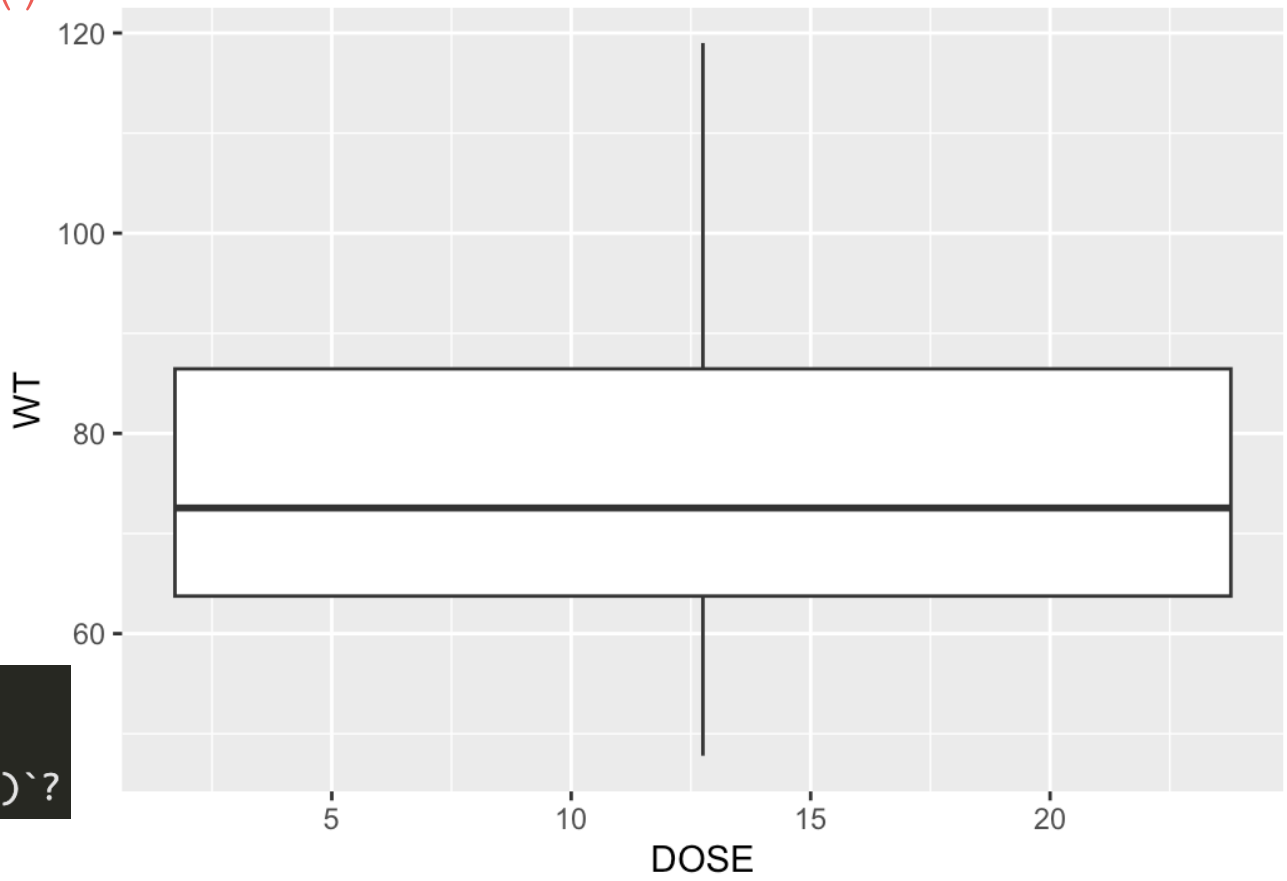


`facet_grid()` also exists which facets by more than one variable

Facets are ordered alphabetically unless they are factors – use factors to enforce ordering of facets

boxplots in ggplot

```r
dat %>%
group_by(ID) %>% slice(1) %>% ungroup() %>%
  ggplot(aes(x=DOSE, y=WT)) +
    geom_boxplot()
```



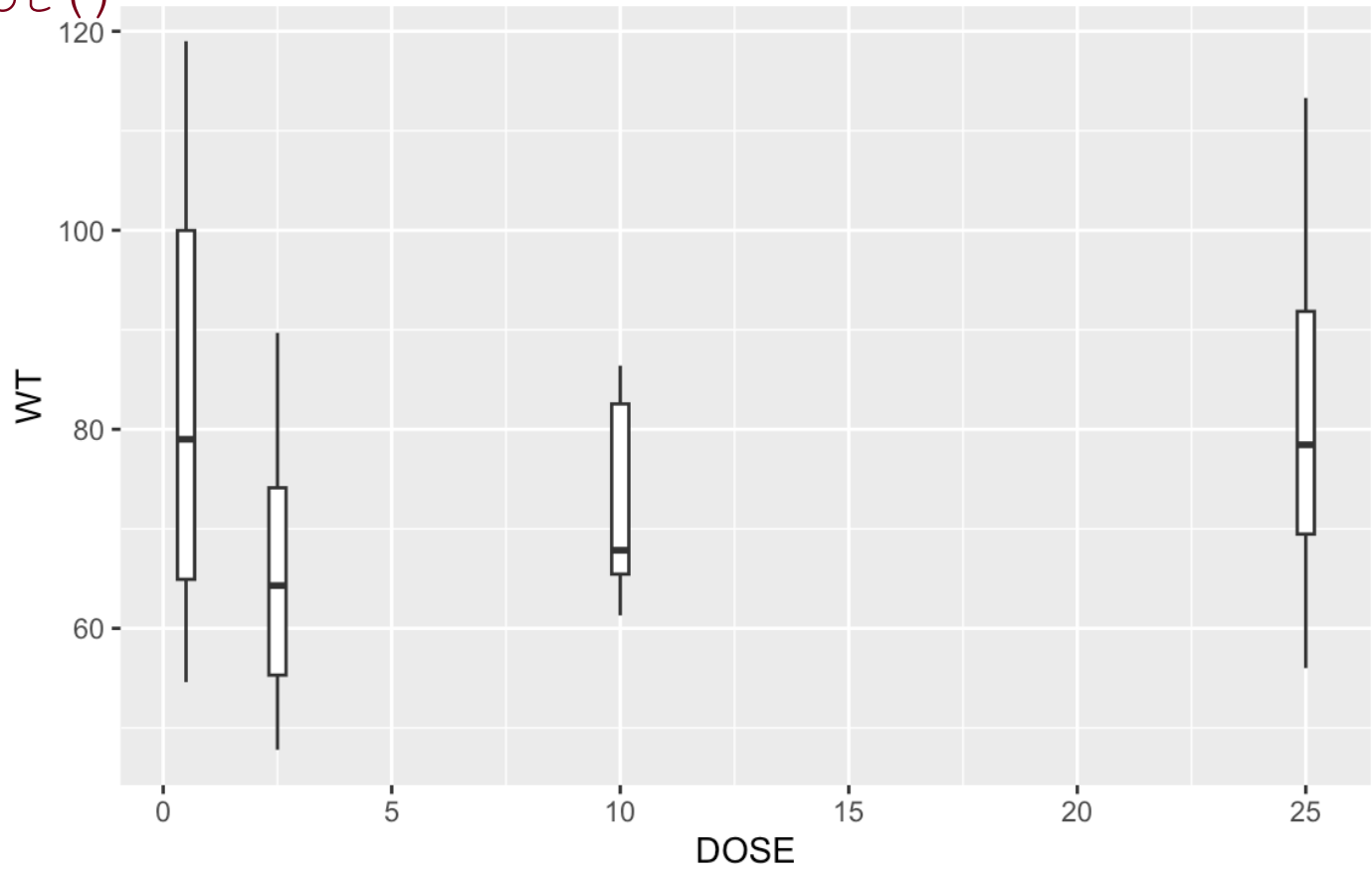
This is not what we want.

Creating boxplots requires the `group` element to be defined to work properly

```
Warning message:
Continuous x aesthetic
i did you forget `aes(group = ...)`?
```

```
dat %>%
  group_by(ID) %>% slice(1) %>% ungroup() %>%
  ggplot(aes(x=DOSE, y=WT, group=DOSE)) +
    geom_boxplot()
```

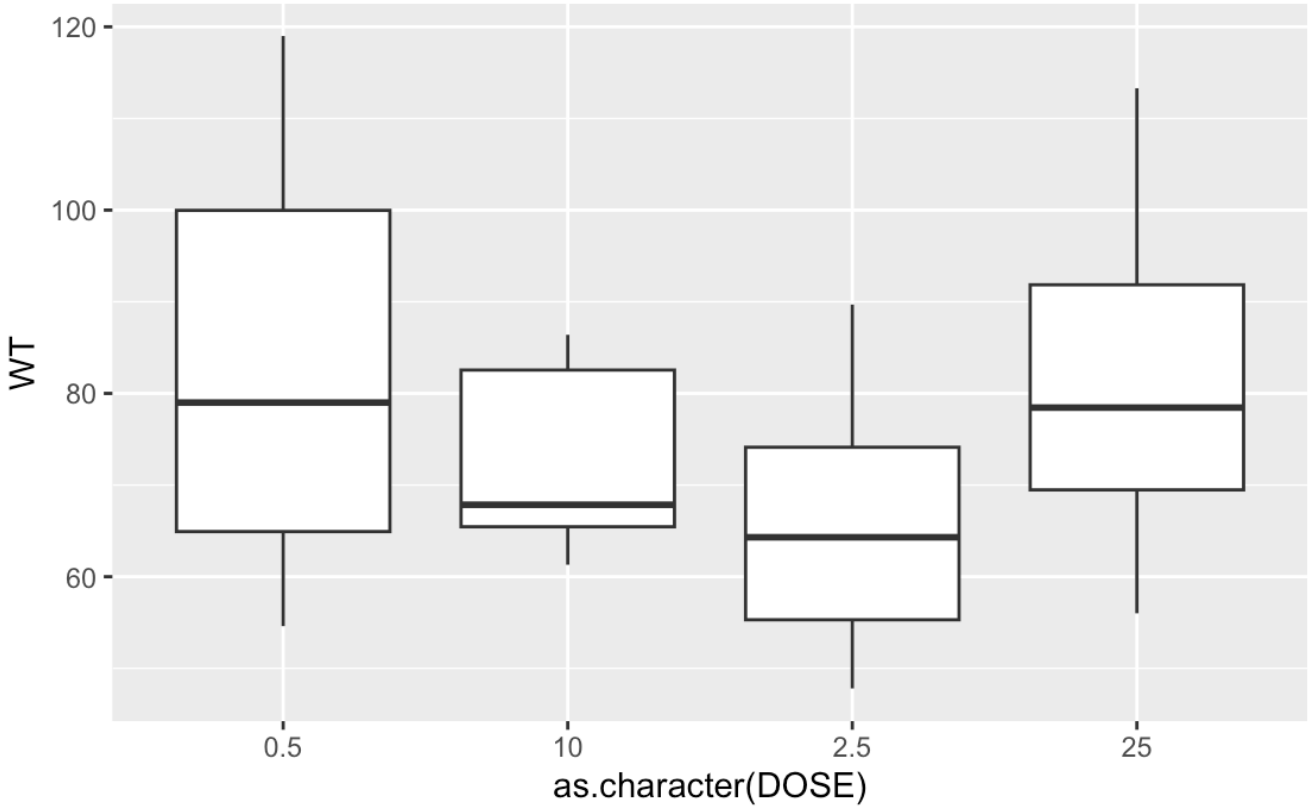
This is getting closer, but why are we seeing the weird spacing on the x-axis?



```
dat %>%
  group_by(ID) %>% slice(1) %>% ungroup() %>%
  ggplot(aes(x=as.character(DOSE),
             y=WT, group=as.character(DOSE))) +
    geom_boxplot
```

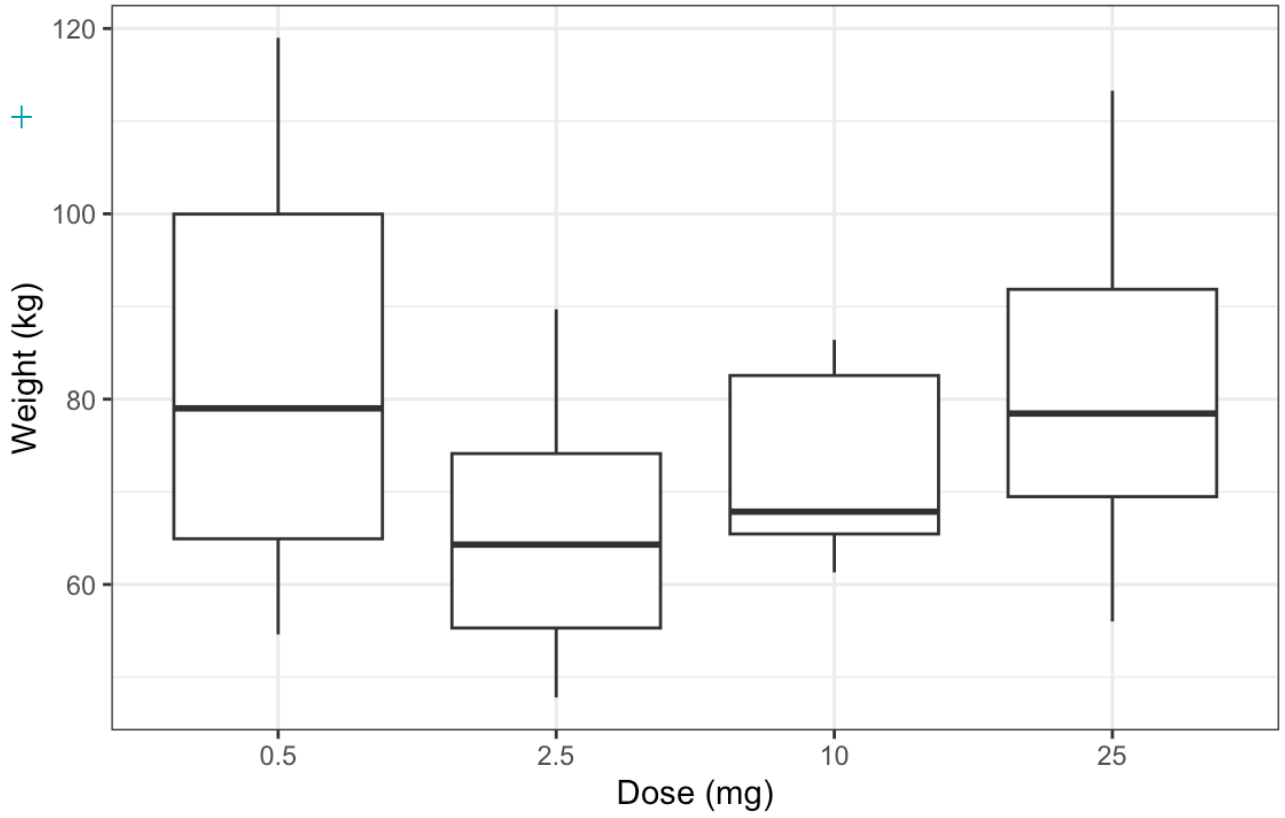
`group` is no longer required when x is a character/factor

Our spacing is fixed, but the order is not correct because it's alphabetical

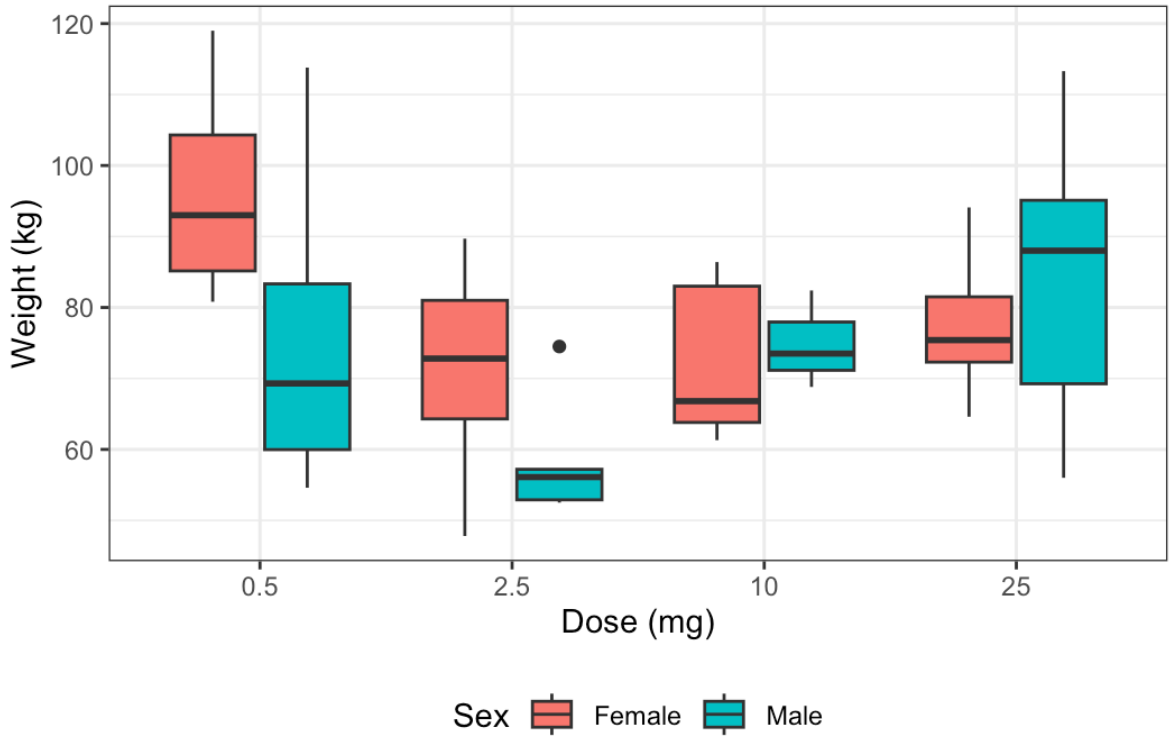


```
dat %>%
  group_by(ID) %>% slice(1) %>% ungroup() %>%
  mutate(DOSE_f = factor(DOSE, levels = c(0.5,2.5,10,25))) %>%
  ggplot(aes(x=DOSE_f, y=WT)) +
    geom_boxplot() +
    labs(x="Dose (mg)",
         y="Weight (kg)") +
    theme_bw()
```

Creating factors enforces order for faceting as well!

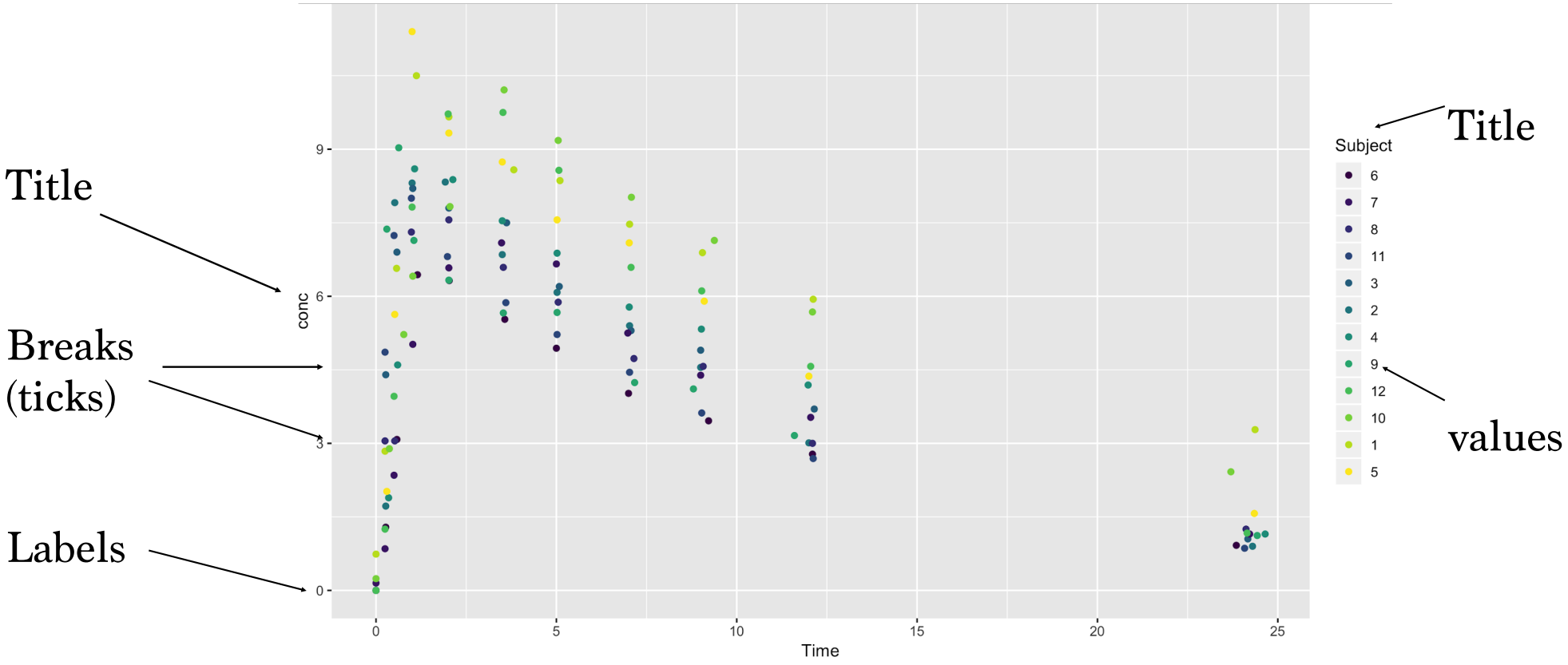


```
dat %>%
 <slice and mutate> %>%
  ggplot(aes(x=DOSE_f, y=WT, fill=SEX_c)) +
    geom_boxplot() +
    labs(x="Dose (mg)", y="Weight (kg)", fill="Sex") +
    theme_bw() + theme(legend.position = "bottom")
```



customizing aesthetics

Scales relate to axis and legends



scale_<aes>_<type>(name, breaks, labels)

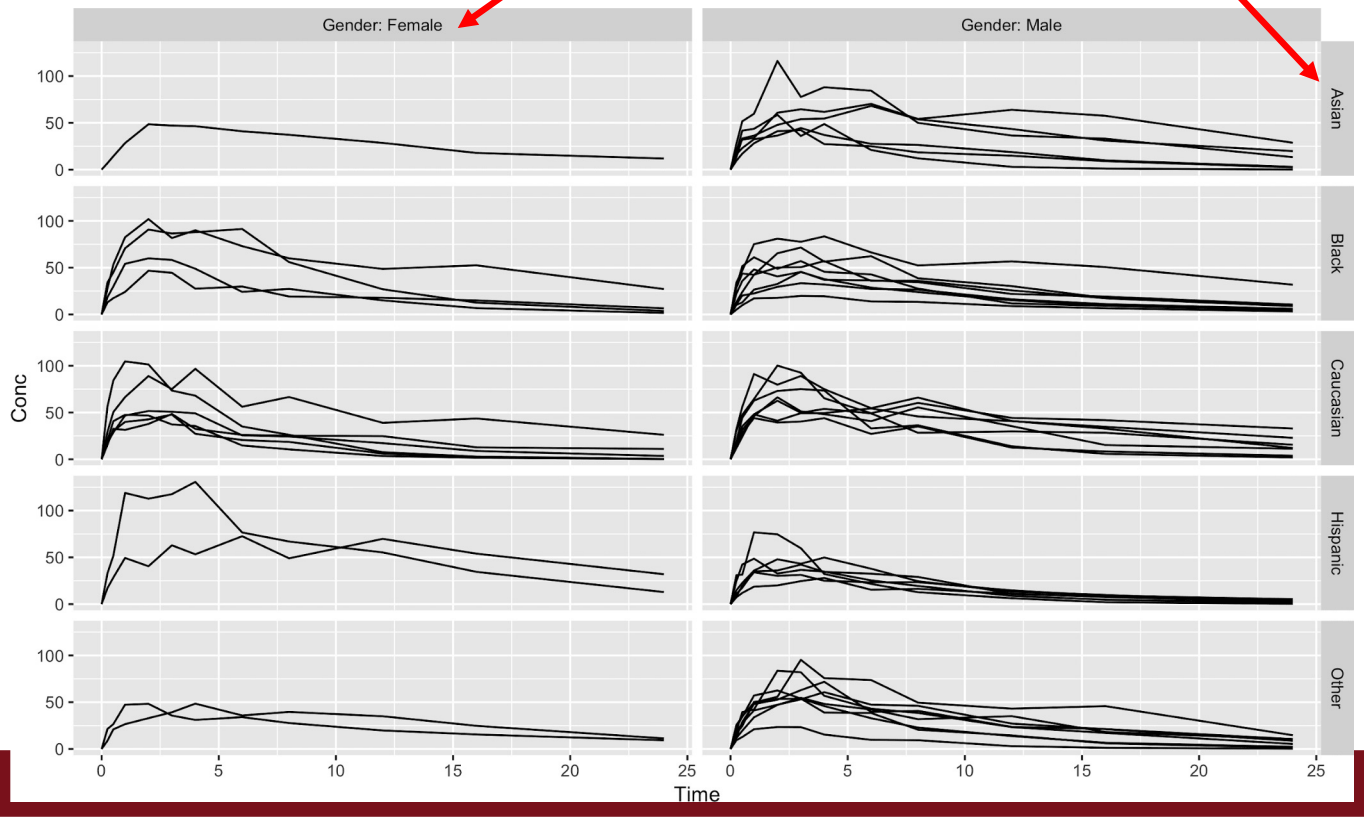
```
ggplot(Theoph, aes(time, conc)) +
geom_point(aes(color = Subject)) +
scale_color_discrete(name = 'ID') +
scale_y_continuous(name = 'Concentration (mg/L)',
breaks = c(0, 0.5, 1, 2, 5, 10))
```

Naming convention for a scale has three elements separated by "_"

1. "`scale`"
2. The name of the aesthetic (e.g., color, shape or etc)
3. The name of the scale (e.g., continuous, discrete, manual).



```
ggplot(sd_oral_richpk, aes(x = Time, y = Conc)) +
geom_line(aes(group = ID)) +
facet_grid(Race~Gender,
labeller = labeller(Gender = label_both, Race = label_value))
```



Summary of ggplot

- Objects are layered in sequential order to create the plot
  - first line = bottom layer, last line = top layer
- Aesthetics and data are inherited from ggplot() object, but can be overwritten within individual geometries (`geom_*`)
- Grouping is important to ensure plots turn out how you want them to look
  - adding aesthetics enforces grouping as well, boxplot can be a bit tricky, just play with it
- There are a lot of helpful tutorials how to make specific plots on the internet, don't hesitate to Google this stuff!



Loops and Functions



Don’t copy-paste your code

- Whenever you get the urge to copy a chunk of code and use it somewhere else in the same script, you should write a function
  - Prevents accidental typos introduced in copy-pasting
  - Anything you would change becomes an argument

- “When will I use this?”
  - Helper functions
  - Repetitive workflows
  - Simulations/bootstraps
  - Plotting

Anatomy of a function in R

```
## function to calculate factorial of x
my_factorial <- function(x){
  ## calculate product for all numbers from 1 to x
  product <- prod(seq(1,x,1))
  ## return product
  return(product)
}

## function test
my_factorial(3)
> 6
```

Function name
Function arguments
Internally-scoped variables
Return statement



A two-argument example

```
## function to implement the "choose" operator
my_choose <- function(x,y){
  ## calculate x choose y using our factorial function
  my_factorial(x) / (my_factorial(y) * my_factorial(x-y))
}

## test function
my_choose(3,2)
> 3
```

If `return()` is not explicitly called, the outputs of the last line will be returned automatically

BUT it is better to be explicit!



Using defaults and ellipsis

```
ci95 <- function(x, lo_q=0.025, hi_q = 0.975, ...){
  lo <- quantile(x, probs=lo_q, ...)
  hi <- quantile(x, probs=hi_q, ...)
  return(c(lo, hi))
}
ci95(1:200)
>  2.5% 97.5%
>  5.975 195.025

ci95(1:200, names=FALSE)
>  5.975 195.025
```



Using function outputs

```
## assign function output to named variable
out_90 <- ci95(1:200, lo_q=0.05, hi_q = 0.95)

out_90
>    5%    95%
> 10.95 190.05

## display output in meaningful context
paste("90% CI:", out_90[1], "-", out_90[2])
> "90% CI: 10.95 - 190.05"
```



Summary of functions

- If your function shares a name with a function already in the environment, the existing function will be overwritten with your new function
  - Bad practice to overwrite existing function names with new functions
- Arguments without default values are required for the function to run
- Arguments with defaults will use the default value unless the function call provides new values
- Ellipsis are used to pass additional arguments to calculations/functions that are called inside your function
- Any object created inside a function cannot be accessed outside of the function (scoping)
- Comments and meaningful variable names are the best way to ensure your function can be reused by other people (and future you!)

The “for” loop

- In programming, a “loop” means a process which is repeated a certain number of times (“for” loop) or until a specific condition is met (“while” loop)
  - We won’t discuss while loops today since they are not used as frequently in data analysis

loop counter variable

```
for(i in 1:n){<operation>}
```

function call

loop sequence



A simple for loop

```
print("ignition sequence")

for (i in 5:1){
  print(i)
}

print("blastoff")
```



Executing code in a for loop

```
vec <- c( "one", "two", "three" )

for ( i in 1:length( vec ) ) {
    if ( vec[ i ] == "one" ) {
        vec[ i ] <- "1"
    }
}
message( "Replaced vector:" )
print( vec )
```

here is an example of a conditional statement which differs from the `if_else()` format shown last week



Vector calculations in R

- For loops are common in most programming languages
- R is built for vector calculations, making many loops unnecessary
- The code on the previous page could be re-written without a loop as such:

```
vec <- c( "one", "two", "three" )
vec [ vec == "one" ] <- "1"
print( vec )
```



apply() functions

- Furthermore, the for loop is slow in R due to its emphasis on vectorized coding
- Base R has a series of functions called the apply() functions which are recommended to be used in place of for loops
  - apply(), lapply(), sapply(), tapply(), vapply()
- These functions run much faster than for loops in most use cases



`map`: how tidyverse implements for loops

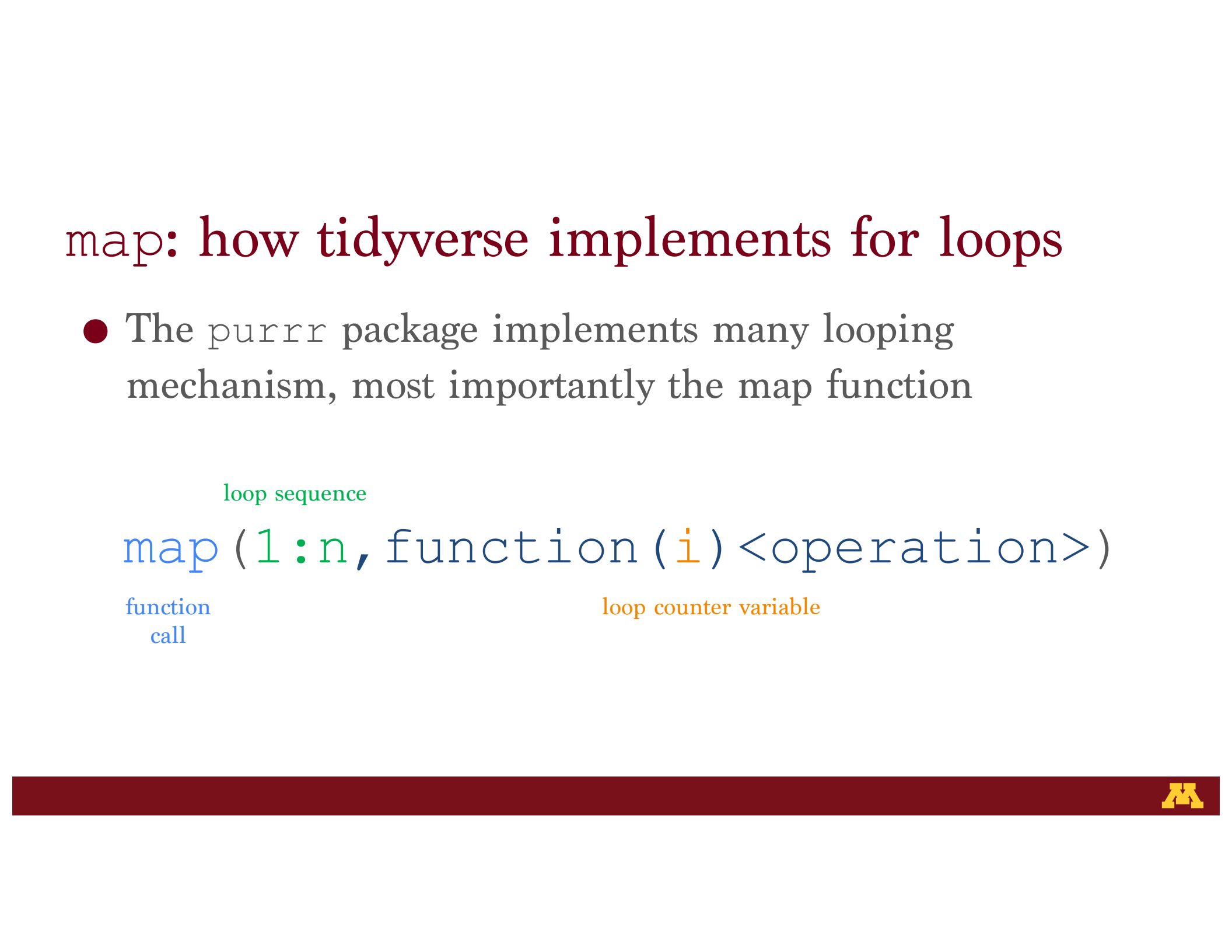
- The `purrr` package implements many looping mechanism, most importantly the map function

loop sequence

```
map(1:n,function(i)<operation>)
```

function call

loop counter variable



Using functions in `map`

```
vec <- c("one", "two", "three")
replace_one <- function(i) if_else( i == "one" , "1", i)
purrr::map(vec, replace_one)
[[1]]
[1] "1"

[[2]]
[1] "two"

[[3]]
[1] "three"
```

map defaults to returning output as a list, but there are several helper functions which will typecast outputs for you automatically



Using functions in `map`

```
replace_one <- function(i) if_else( i == "one" , "1", i)
purrr::map(c("one", "two", "three"), replace_one)
[[1]]
[1] "1"

[[2]]
[1] "two"

[[3]]
[1] "three"
```

You don't need to pass named objects into your functions; you can pass in objects you create on the fly



Typecasting output and anonymous functions

```
> out <- purrr::map_chr(vec, function(i) {
+   if_else( i == "one" , "1", i)
+ })
> class(out)
[1] "character"
> out
[1] "1"     "two"   "three"
```



Simulating a small clinical trial population

```
sim_trial_arm <- function(x, n_subjects=4) {
  data.frame(
    ARM = x,
    ID = seq(n_subjects),
    DOSE = <some function>,
    MALE = <some function>,
    WT = <some function>
  ) %>% return()
}
```



Simulating a single arm

```
> sim_trial_arm(1)
  ARM ID     DOSE MALE    WT
    1  1    10 mg    1 100.3
    1  2    10 mg    1  94.7
    1  3    25 mg    0  83.3
    1  4 Placebo    0  67.4
```

Simulating multiple arms using `map()`

```
> map_df(seq(3), sim_trial_arm)
   ARM ID     DOSE MALE   WT
1    1  1    25 mg    1 84.6
2    1  2 Placebo    1 74.0
3    1  3    10 mg    1 76.0
4    1  4    25 mg    0 67.8
5    2  1    10 mg    1 89.5
6    2  2    10 mg    1 78.2
7    2  3 Placebo    1 77.3
8    2  4    25 mg    1 69.2
9    3  1    25 mg    1 65.4
10   3  2    25 mg    0 82.2
11   3  3 Placebo    0 78.3
12   3  4    25 mg    0 80.1
```

```
> map_df(seq(3), sim_trial_arm)
   ARM ID     DOSE MALE    WT
1    1  1    25 mg    0  69.2
2    1  2    25 mg    0  88.0
3    1  3 Placebo    0  83.6
4    1  4 Placebo    1  80.9
5    2  1    25 mg    1  96.3
6    2  2    10 mg    0  72.8
7    2  3 Placebo    0 106.2
8    2  4    10 mg    1  98.4
9    3  1 Placebo    1  75.8
10   3  2    10 mg    1  68.6
11   3  3    25 mg    0  79.5
12   3  4    25 mg    1  70.6
```

But our results aren't reproducible!



An aside on random numbers in R

- A random number generator is used for any function in R that requires some amount of stochasticity (e.g. sampling from a distribution)
- The `set.seed()` function is one way to setting initial value in random number generator so that reproducible results can be obtained
- Examples of commonly used random functions in R
  - Sample from uniform distribution: `runif(n=5, min=0, max=1)`
  - Sample from normal distribution: `rnorm(n=5, mean=0, sd=1)`
  - Sample from an object in R: `sample(c("red", "blue"), x=5, replace=TRUE)`



Simulating a small clinical trial population

```
sim_trial_arm <- function(x, n_subjects=4) {
  data.frame(
    ARM = x,
    ID = seq(n_subjects),
    DOSE = sample(c("10 mg", "25 mg", "Placebo"),
                  size=n_subjects, replace=TRUE),
    MALE = runif(n_subjects) %>% round(),
    WT = rnorm(n_subjects, mean=80, sd=12) %>% round(digits=1)
  )
}
```



Setting RNG seed allows results to be reproduced

```
> set.seed(123)
> map_df(seq(3), sim_trial_arm)
   ARM ID    DOSE MALE    WT
1    1  1 Placebo    1  81.6
2    1  2 Placebo    0 100.6
3    1  3 Placebo    1  85.5
4    1  4   25 mg    1  64.8
5    2  1   10 mg    1  86.6
6    2  2 Placebo    1  82.9
7    2  3 Placebo    1  67.4
8    2  4   10 mg    1  95.5
9    3  1   25 mg    0  77.4
10   3  2   10 mg    0  67.7
11   3  3 Placebo    0  71.3
12   3  4 Placebo    0  72.5
```

```
> set.seed(123)
> map_df(seq(3), sim_trial_arm)
   ARM ID    DOSE MALE    WT
1    1  1 Placebo    1  81.6
2    1  2 Placebo    0 100.6
3    1  3 Placebo    1  85.5
4    1  4   25 mg    1  64.8
5    2  1   10 mg    1  86.6
6    2  2 Placebo    1  82.9
7    2  3 Placebo    1  67.4
8    2  4   10 mg    1  95.5
9    3  1   25 mg    0  77.4
10   3  2   10 mg    0  67.7
11   3  3 Placebo    0  71.3
12   3  4 Placebo    0  72.5
```



Using `withr::with_seed()`

```
sample(c("red","blue"), 5, replace=TRUE)
[1] "red" "blue" "blue" "red" "blue"

sample(c("red","blue"), 5, replace=TRUE)
[1] "red" "blue" "blue" "blue" "red"

withr::with_seed(1901, sample(c("red","blue"), 5, replace=TRUE))
[1] "red" "blue" "red" "blue" "red"

withr::with_seed(1901, sample(c("red","blue"), 5, replace=TRUE))
[1] "red" "blue" "red" "blue" "red"
```



Adjusting arguments using the tilde operator

```
> purrr::map_df(seq(3), ~ sim_trial_arm(.x, n_subjects = 6))
   ARM ID    DOSE MALE   WT
1    1  1   10 mg    1 90.7
2    1  2 Placebo    0 90.5
3    1  3   10 mg    1 89.9
4    1  4 Placebo    0 88.3
5    1  5   25 mg    0 86.6
6    1  6   10 mg    0 79.3
7    2  1 Placebo    1 75.2
8    2  2   25 mg    1 74.4
9    2  3   25 mg    1 89.4
10   2  4 Placebo    0 79.0
11   2  5   25 mg    0 83.0
12   2  6   25 mg    1 79.7
13   3  1   25 mg    1 84.6
14   3  2   10 mg    0 74.0
15   3  3 Placebo    1 76.0
16   3  4 Placebo    1 67.8
17   3  5   25 mg    1 67.1
18   3  6 Placebo    0 83.6
```

The tilde operator ~ is equivalent to `function(.x)`



Homework Part 1: Plotting our data

- Using the `mad-nonmem.csv` dataset again, create the following plots:
- Plot 1: boxplot showing the distribution of age **for each sex** in your data set
  - hint: consider the number of rows per subject
- Plot 2: spaghetti plot (e.g. a line for each subject and a dot for each observation) of all subjects with each **dose group** uniquely colored, faceted by study day (DAY column), with the y-axis presented on a log scale
  - hint: use Factors to get doses to show up in order
- Plot 3: use summarize() and create a boxplot of individual Cmax values for each dose level

Homework Part 2: Writing a custom summary function

- Import mad_nonmem.csv
- Write a function that summarizes continuous covariates and outputs them in the following format: "`mean [5th percentile, 95th percentile]`"
- Apply the function separately for each dose group across the following covariates
  - `WT, AGE, EGFR, BILI`
- Pivot the data set so you have three columns: Dose, Covariate Name, and Covariate Summary Value
- Use `map()` to create a list of individual concentration-time plots